The eqsat Substrate: Equality Saturation with Replayable Certificates for Learned Rewrite Games

Draft — proposed, not accepted

mlopt project

2026-10-21

Abstract

Learned play over program rewriting needs a world: terms a policy can move, a referee that can check the moves, a score that can rank them, and an exit that can run the result. We describe eqsat — the substrate under the mlopt game — as one artifact. A term IR and a bounded e-graph saturate a program under a law bank; every delivered answer carries a replayable derivation certificate — steps the e-graph witnessed but that have no standalone justification are flagged `egraph_dependent`, and `verify_certificate(strict=True)` accepts only fully standalone certificates; a rewrite law is a serializable data row — a pattern pair plus declarative `cond`/`dspec` guards and procedural `check`/`derive` hooks; machinery mints composite laws by fixpoint and ships them through a staged admission gauntlet that has falsified the engine's own candidates; a cost algebra prices shared structure honestly so a fold's compile-time work is not billed per call; and a hexagonal port surface lets backends lower, execute, and verify without the engine importing them. Because the pieces are substrate, novelty becomes a decidable predicate — certified, strictly paying, and underivable from the catalogued bank — rather than a literature claim. The repository is `handl-lang/eqsat`; the domain consumers are the tensor-optimization paper (`handl-lang/graphopt`) and a private codegen-domain companion.

> **!** Important
>
> **Status.** This is a working draft — *proposed, not accepted*. It has not had human revision or external review. It is the platform paper: it owns the substrate narrative once so the domain papers can cite it instead of repeating it, and every claim names the module or artifact that carries it.

1 Introduction

Every learned-rewriting system assembles the same four pieces — a term world, a law bank, a referee, a score — and every paper about one re-explains them. The mlopt project split that world out: `handl-lang/eqsat` is the substrate repository, and the domain repos consume it as pinned dependencies on `packages/` subdirectories. This paper is the substrate's own account — deliberately compact, because its job is to be cited, not to report a domain result.

The pieces have arrived separately. Massalin's superoptimizer (Massalin 1987) searched programs exhaustively; equality saturation (Tate et al. 2009; Willsey et al. 2021) closes the program space under a fixed law bank. Rule-inference work (Ruler (Nandi et al. 2021), TASO (Jia et al. 2019), the Enumo line (Pal et al. 2023)) grows the bank itself. And learned play on well-defined games —

AlphaGo Zero (Silver et al. 2017), AlphaTensor (Fawzi et al. 2022), AlphaGeometry (Trinh et al. 2024) — reaches structure beyond hand-written heuristics. eqsat is the layer at which these stops being separate: one calculus holds the programs, the laws, and the derivations — the cells-as-data pattern the handl kernel defines (handl project 2026), realised here in stdlib-only Python that imports nothing from the kernel — so a discovered rule re-enters the bank it was found with — a value, not a build step.

The claim is narrow and checkable:

> The substrate is *self-contained*: programs, rewrite laws, and derivations are data in one engine — serializable, storable, composable — and every optimized artifact it ships carries a proof the consumer can replay without trusting the search that made it.

No domain speedup is claimed here; those claims, and their measured ledgers, belong to the domain papers (Section 8). What this paper fixes is the shared vocabulary: the e-graph and its bounds (Section 2), laws and their guard contract (Section 3), certificates and bounded derivability (Section 4), the ports (Section 5), the cost algebra (Section 6), and the falsifiable novelty predicate (Section 7).

2 Terms and the e-graph

`mlopt_core.ir` is the term language — `Op`, `Var`, `Param`, `Const`, `TensorType`, with attrs carried as plain data. A program reads as a 1-morphism between tensor types; a rewrite law is a 2-cell between programs; a derivation — one law proven from premises — is a 3-cell (the cells-as-data framing of the kernel calculus (handl project 2026), implemented here directly — eqsat is pure Python and runs on none of the kernel's combinators). The operational consequence is that all three levels are values the same engine manipulates.

`EGraph.add_term` interns a term; an `ENode` is an operator plus children e-class ids, hash-consed so identical subterms share storage, and a union-find merges the classes a rewrite proves equal. `EGraph.run` is then a *closure*, not a search: it fires the rule bank to a fixed point, and the e-graph *is* the set of consequences. Its controls are honest bounds — `max_iterations`, `max_nodes`, `rule_budgets`, `stop`, `patience` — and a bound can only ever refuse a claim: resource exhaustion reports unfinished, never a fabricated merge. `extract_best` picks the cheapest member of a saturated class under a cost function. At the pipeline seam one level up, `search`/`Optimizer`'s `engine=` kwarg admits an optional Rust engine (`mlopt-native`, maturin/PyO3) — opt-in, never auto-detected, with `stats["engine"]` on the `SearchResult` recording which ran (Section 5).

3 Laws as data, and the machinery over them

A `Rewrite` is a data row: an LHS/RHS pattern pair plus a guard spec. Guards come in two vocabularies that conjoin — `cond`/`dspec` are *declarative* (pure data in the `laws.cond` grammar, evaluated against the match binding), while `check`/`derive` are procedural hooks for what the declarative fragment cannot express. This is the soundness boundary: any unsound rule poisons every e-class it touches, so a rule's side conditions are the *only* layer that keeps a false member out — certificate replay (Section 4) re-proves that a step is a valid application, not that the rule is true.

The shipped bank — `mlopt_core.laws.ALL_RULES` (86 rules in this tree), the `DEFAULT RuleSet`, the `prior_art` preset — mixes human-authored axioms with thirteen machine-admitted objects (`mlopt_core.laws.provenance.MACHINE_SHIPPED`). `laws.serialize` gives each rule a content key

(`alpha_key` up to metavariable renaming), which is what makes "the same law, spelled again" a decidable question.

Three pieces of machinery operate on the bank:

Minting. `mlopt_core.meta.synthesize_rules` runs a bounded fixpoint over the bank on a seed term: each round applies rules, composes the pairs that chain, and emits `syn_*` composites carrying *premise provenance* — the names of the rules each composite was built from.

Guard transport. A minted rule's side conditions are closures over its premise objects, so they are stored as *specs* — parent names plus re-expression maps — not code, and `check`/`derive` are rebuilt by resolving parents transitively. `mlopt_core.laws.minted` embeds those records as data and replays the same rebuild at import, louder than the store: a parent resolving short of declared strength raises rather than admitting a degraded rule.

Admission. A staged gauntlet decides what re-enters the bank: reconstruct → full-data → measure → truth → novelty → typed-pay → closure → certificate. The measure gate carries both the numeric oracle *and* premise-anchored derivability; the truth gate is the guarded-region sweep — every guard-accepted evaluable binding must be equal, and at least one must exist — and novelty refuses re-spellings of shipped laws. The staging policy lives in the discovery domain — `mlopt_discovery.evidence` in `handl-lang/graphopt` — but every leg it calls is substrate: guard rebuild, saturation, certificate replay, alpha-keyed duplicate detection, typed extraction. It has repeatedly falsified the engine's own candidates; the falsification record is the domain paper's (mlopt project 2026).

4 Certificates and bounded derivability

A certificate is the artifact a search produces that outlives the search. Saturation records each merge as a `ProofEdge` — the fired rule, the two canonical e-class ids, the substitution — and `EGraph.certificate(src, dst)` reconstructs from that merge log an ordered list of `CertStep`s, positional term-level steps `(rule, path, lhs, rhs, bindings, egraph_dependent, note)` that locate each rewrite by a child-index `path`; no e-class survives into the step. `verify_certificate(term, cert)` replays the derivation standalone: descend to `path`, re-match the rule's LHS against the subterm there, check the recorded bindings and re-run the `check`/`derive` side conditions, instantiate the RHS, substitute — and require the reconstructed term to equal `cert.dst` exactly; a certificate that does not replay raises rather than passing. Steps the e-graph witnessed but that have no standalone justification (`egraph_dependent` — non-local merges such as the pairing pass, manual unions, budget exhaustion) replay as trusted assertions under the default `strict=False` and are rejected under `strict=True`. The codec (`cert_to_data`/`cert_from_data`) makes the certificate a wire value: it can cross a cache or a backend boundary.

Certificates do two jobs. *Delivery*: every program the pipeline lowers ships its certificate — search → extract → certify — so downstream consumers verify the derivation, not the optimizer's reputation. *Admission*: `verify_law` answers "is this instance derivable under a rule bank?" with a replayable witness — intern both sides in a fresh small e-graph, saturate under the known laws, replay the witness in both directions. For a stored composite the honest question is derivability *under the premises it was minted from*, so the check evaluates against `base_rules` plus the object's transitive premise closure — with an anti-gaming bar: the certificate must be standalone-replayable and every step naming a non-axiom rule must site-verify under the fp64 numeric oracle. A premise false at its applied site revokes the credit and the verdict falls back to the axioms-only answer. `verify_law`

itself is domain-side (`mlopt_discovery.verifier`); everything it calls is here.

The boundary is honest: a certificate proves *equality of the derivation*, nothing else — whether the extraction was worth it is a measured question the certificate deliberately does not answer.

5 The ports

eqsat never imports a backend, and a backend never sees the search. They meet through `mlopt_core.ports`: `Protocol` shapes each side implements or consumes.

- **`Source / Sink`** — lift a domain object into terms, and the way back down: `lower` emits a runnable artifact, `verify` checks it against the source, `executors` lists what can run it, all on top of `Capabilities`' hard feasibility declaration. A certified result is not married to the backend that priced it.
- **`Engine`** — the `engine=` contract: an alternate saturation engine, passed explicitly, never auto-detected.
- **`Policy`** — *orders* the saturation schedule. Every rule still fires, so the fixed point and the certificate are unchanged: a policy can never decide equivalence.
- **`Runner / Executor / Composer / Criterion / CostFn / Strategy`** — delivery and choice: how a lowered artifact is executed, timed, chained, and what "cheapest" means.

The port binds shapes, never contents: the bank rows, the domain registries, and the weight tables are data — the names are frozen (the contract lives upstream in `graphopt:docs/public_api.md`; this repo's `docs/site/reference/frozen-api.md` is the vendored eqsat slice of that manifest), the rows are each domain repo's business.

6 The cost algebra

`dag_cost(term, cost_fn)` prices a program graph honestly: shared subtrees are paid once, so a DAG is not billed as a tree. It sums static weights — FLOPs per op, parameter bytes, node counts — with no tensors ever executing. One structural discount matters: a subtree closed over `Param`/`Const` leaves whose root op is in `FOLD_OPS` — the 161-op fold-at-lowering alphabet, single-sourced at the substrate and consumed by the domain lowerers — is priced at ~0 marginal, because it is once-per-deployment work, not per-call work. The shipped models cover counting (`count_cost`, `flops_cost`, `param_bytes_cost`), backend-aware (`backend_cost`, `executor_cost_for`, `fused_cost_for`), and shape-aware (`depth_cost`, `roofline_cost`, fusion regions) axes.

`dag_cost` has no notion of memory traffic, kernel fusion, or launch overhead, so it *ranks* candidates — measured lanes (fixtures, verify, wall clocks) make the claims. The game-layer consequence is what matters here: a cheap, deterministic, checkable score that a policy can be trained against without executing a tensor.

7 The falsifiable novelty predicate

"Novel" is where rewrite-discovery systems go vague. The substrate makes it decidable — *machine-original against catalogued knowledge*, a closed-world claim with a counter-witness on every leg. A minted law is a novelty candidate iff three gates pass:

- **cert** — the candidate's own rule derives its instance with a standalone-replayable certificate. A rule whose own rewrite cannot carry a replayable witness fails here.

- **beat** — on at least one registered baseline case, the certified extraction under `rules + {candidate}` lands *strictly below* the best recorded baseline — a tie is a rediscovery, not a find — and the derivation certificate actually invokes the candidate. A win the rule never touched is not the rule's win.
- **¬derivable** — the candidate's instance is not saturatable under the known bank with the candidate *and every alpha-sibling* (`laws.serialize.alpha_key`) removed. "Derives anyway" means the bank already proves it: the law adds no theorem. An inverse spelling is a different law and stays in the premises — a bank holding both directions honestly knows the equation.

The predicate is implemented by the discovery domain's sieve (`bench/drivers/screen/novelty_sieve.py` in `handl-lang/graphopt`), whose `known_bank()` defaults to the widest shipped surface the bounded prover can saturate against — shipped rules plus the `prior_art` preset's `FUSION/CATEGORICAL` rows (the `SIMPLIFICATION/EXPANSIVE` families explode the derivability checker's node budget and stay out of the floor) — so derivability is tested against the full reachable slice of catalogued knowledge. What it does *not* claim is "novel to humanity": that remains an open-world literature question, and the domain papers say so explicitly.

8 The domain papers

The substrate is the shared basis of two companion drafts:

- **The discovery domain** — `handl-lang/graphopt` reports one complete traversal of this machinery (mlopt project 2026): a rewrite a hand optimizer would write (the eval-mode conv+BN weight fold) minted, admitted, shipped verbatim in `MACHINE_SHIPPED`, and runtime-priced bitwise-equal through an independent engine.
- **The codegen domain** — a private codegen-domain companion paper applies the same substrate to fused-kernel law discovery; it is cited here only to bound the platform story, not for results.

A third sibling, `handl-lang/gamekit`, is the orthogonal game layer — the `Board` protocol, episode/train/eval drivers, and the player registry — substrate-free by contract (`gamekit` $\perp$ `eqsat`, enforced by import-linter): the domain repos implement its board over the machinery described here, which is what makes "a learned policy plays the rewrite game" a well-formed sentence — moves are rule applications, the gauntlet is the referee, the cost algebra is the score.

9 Related work

Equality saturation. Tate et al. (Tate et al. 2009) introduced the technique; egg (Willsey et al. 2021) made it practical; TENSAT (Yang et al. 2021) applies it to tensor graphs; egglog (Zhang et al. 2023) unifies it with Datalog — and serves the mlopt project as a test-only differential oracle. eqsat's difference is not the saturation loop but what surrounds it: certificates as first-class transportable values, and a bank that the discovery loop writes back into.

Rule inference and library learning. Ruler (Nandi et al. 2021) infers rewrite rules by enumeration plus equality-saturation filtering — the direct ancestor of mint-then-verify. TASO (Jia et al. 2019) generates DNN substitutions and verifies them with a theorem prover; Isaria (Thomas and Bornholt 2024) synthesizes sound rule systems from ISA specifications; Enumo (Pal et al. 2023) makes rule inference programmable. Each grows a bank offline; in eqsat the minted object is a value in the runtime calculus, so admission is a runtime decision, not a build step. Babble (Cao et al. 2023) and Glenside (Smith et al. 2021) are the library-learning and access-pattern precedents respectively.

Learned play over rewriting. CompilerGym (Cummins et al. 2022) casts compiler optimization as an RL environment; Omelette (Singh 2022), MCTS-GEB (He, Singh, and Yoneki 2023), and Aurora (Bărbulescu et al. 2024) train policies that steer e-graph construction — over a static bank. AlphaGeometry (Trinh et al. 2024) couples a learned proposer to a certifying deducer, and AlphaTensor (Fawzi et al. 2022) searches one fixed structure. The differentiator here is the closure: a law minted during play re-enters the bank as a move.

10 Limitations and availability

- **Language and scale.** The reference engine is pure-Python stdlib; `mlopt-native` (PyO3/Rust) is opt-in and covers search only — certificates and the non-local passes are Python-engine capabilities.
- **Certificates bound, not arbitrate.** Replay proves equality of derivation under the given bank; it does not prove the bank sound. The guard contract (Section 3) is the only layer between a wrong rule and a poisoned e-class.
- **Not every certificate is fully standalone.** A derivation can carry `egraph_dependent` steps — merges the e-graph witnessed that have no positional rewrite justification — which replay as trusted assertions (their recorded `lhs` must still match) under the default `strict=False`; only `verify_certificate(strict=True)` rejects them (Section 4). The certificate is honest about the flag in the data, but a consumer running default verification is trusting those steps.
- **Cost is a ranker.** `dag_cost` shares the limits of every static model; measured lanes remain the arbiter of any speedup claim.
- **Novelty is closed-world** by construction (Section 7) — a design choice, honestly named, not a literature-survey result.

The repository is `handl-lang/eqsat`. Everything named above is importable from the committed tree:

```
from mlopt_core.egraph import EGraph, verify_certificate
from mlopt_core.laws import ALL_RULES, DEFAULT
from mlopt_core.laws.provenance import MACHINE_SHIPPED
from mlopt_core.meta import synthesize_rules
from mlopt_core.cost import dag_cost
from mlopt_core.ports import Sink, Source
from mlopt_orchestrator import search, lower, Optimizer
```

References

Bărbulescu, George-Octavian, Taiyi Wang, Zak Singh, and Eiko Yoneki. 2024. "Learned Graph Rewriting with Equality Saturation: A New Paradigm in Relational Query Rewrite and Beyond." *arXiv Preprint arXiv:2407.12794*.

Cao, David, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. "Babble: Learning Better Abstractions with e-Graphs and Anti-Unification." *Proceedings of the ACM on Programming Languages* 7 (POPL): 396–424. https://doi.org/10.1145/3571207.

Cummins, Chris, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, et al. 2022. "CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research." In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation*

and Optimization (CGO), 92–105. IEEE. https://doi.org/10.1109/CGO53902.2022.9741258.

Fawzi, Alhussein, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. 2022. "Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning." *Nature* 610: 47–53.

handl project. 2026. "The Handl Kernel: A Closed Arrow Calculus for Composable Languages."

He, Guoliang, Zak Singh, and Eiko Yoneki. 2023. "MCTS-GEB: Monte Carlo Tree Search Is a Good e-Graph Builder." In *Proceedings of the 3rd Workshop on Machine Learning and Systems (EuroMLSys)*. ACM. https://doi.org/10.1145/3578356.3592577.

Jia, Zhihao, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. "TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions." In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 47–62. ACM. https://doi.org/10.1145/3341301.3359630.

Massalin, Henry. 1987. "Superoptimizer: A Look at the Smallest Program." In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, 122–26. IEEE Computer Society Press.

mlopt project. 2026. "Certified Rewrite Discovery for Tensor Programs: A Machine-Minted Conv+BN Fold from Minting to Runtime."

Nandi, Chandrakana, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. 2021. "Rewrite Rule Inference Using Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (OOPSLA): 1–28. https://doi.org/10.1145/3485496.

Pal, Anjali, Brett Saiki, Ryan Tjoa, Cynthia Richey, Amy Zhu, Oliver Flatt, Max Willsey, Zachary Tatlock, and Chandrakana Nandi. 2023. "Equality Saturation Theory Exploration à La Carte." *Proceedings of the ACM on Programming Languages* 7 (OOPSLA2). https://doi.org/10.1145/3622834.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550: 354–59. https://doi.org/10.1038/nature24270.

Singh, Zak. 2022. "Deep Reinforcement Learning for Equality Saturation." Master's thesis, University of Cambridge.

Smith, Gus Henry, Andrew Liu, Steven Lyubomirsky, Scott Davidson, Joseph McMahan, Michael Taylor, Luis Ceze, and Zachary Tatlock. 2021. "Pure Tensor Program Rewriting via Access Patterns." In *Proceedings of Machine Learning and Systems (MLSys)*, 3:273–86.

Tate, Ross, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. "Equality Saturation: A New Approach to Optimization." In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 264–76. ACM. https://doi.org/10.1145/1480881.1480915.

Thomas, Samuel, and James Bornholt. 2024. "Automatic Generation of Vectorizing Compilers for Customizable Digital Signal Processors." In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 19–34. ACM. https://doi.org/10.1145/3617232.3624873.

Trinh, Trieu H., Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. "Solving Olympiad Geometry Without Human Demonstrations." *Nature* 625: 476–82. https://doi.org/10.1038/s41586-023-06747-5.

Willsey, Max, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. "Egg: Fast and Extensible Equality Saturation." *Proceedings of the ACM on Programming Languages* 5 (POPL): 1–29. https://doi.org/10.1145/3434304.

Yang, Yichen, Phitchaya Mangpo Phothilimthana, Yisu Remy Wang, Max Willsey, Sudip Roy, and

Jacques Pienaar. 2021. "Equality Saturation for Tensor Graph Superoptimization." In *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 3.

Zhang, Yihong, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2023. "Better Together: Unifying Datalog and Equality Saturation." *Proceedings of the ACM on Programming Languages* 7 (PLDI): 468–92. https://doi.org/10.1145/3591239.